

LAPORAN PRAKTIKUM TUGAS
MATERI 5 :
SINKRONISASI SISTEM OPERASI
(SISTEM BOOKING HOTEL)



Dosen Pengampu :
Triawan Adi Cahyanto M.Kom

Disusun Oleh :
MUHAMMAD RIDHO 2410651009 (A)

Tanggal : 9-11-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

Daftar Isi

1. Pendahuluan	
1.1 Latar Belakang	
1.2 Tujuan Praktikum	
1.3 Rumusan Masalah	
2. Dasar Teori.....	
2.1 Konsep Multithreading.....	
2.2 Sinkronisasi dan Race Condition	
2.3 Semaphore dan Mutex.....	
3. Soal Pemrograman: Sistem Booking Hotel	
3.1 Analisis Masalah	
3.1.1 Pemahaman tentang Soal	
3.1.2 Identifikasi Masalah Sinkronisasi	
3.1.3 Desain Solusi.....	
3.2 Implementasi	
3.2.1 Penjelasan Algoritma	
3.2.2 Flowchart / Diagram	
3.2.3 Potongan Kode Penting dengan Penjelasan.....	
3.2.4 Alasan Pemilihan Mekanisme Sinkronisasi	
3.3 Testing dan Output	
3.3.1 Screenshot Output	
3.3.2 Test Case yang Diuji	
3.3.3 Hasil yang Didapat.....	
3.4 Analisis Hasil.....	
3.4.1 Evaluasi Kinerja Program	
3.4.2 Masalah yang Ditemui dan Solusinya	
3.4.3 Performa Program	
3.4.4 Pelajaran yang Didapat	
4. Kesimpulan dan Saran	
4.1 Kesimpulan.....	
4.2 Saran	
5. Daftar Pustaka	

1. Pendahuluan

1.1 Latar Belakang

Dalam dunia komputasi modern, banyak sistem yang berjalan secara paralel dan multithreaded, di mana beberapa proses atau thread dijalankan secara bersamaan untuk meningkatkan efisiensi dan kecepatan eksekusi. Namun, kondisi ini dapat menimbulkan permasalahan baru, terutama ketika beberapa thread harus mengakses sumber daya yang sama secara bersamaan. Permasalahan tersebut dikenal dengan istilah race condition, yaitu situasi ketika hasil akhir suatu program bergantung pada urutan eksekusi thread yang tidak terprediksi.

Untuk mencegah terjadinya race condition, diperlukan mekanisme sinkronisasi yang dapat mengatur akses terhadap sumber daya bersama (shared resource). Dua konsep penting dalam sinkronisasi adalah Semaphore dan Mutex (Mutual Exclusion).

Semaphore digunakan untuk membatasi jumlah thread yang dapat mengakses suatu sumber daya, sedangkan Mutex digunakan untuk melindungi bagian kode kritis agar hanya satu thread yang dapat mengeksekusinya pada satu waktu.

Dalam praktikum ini, dilakukan simulasi Sistem Booking Hotel menggunakan konsep thread, semaphore, dan mutex. Melalui simulasi ini, mahasiswa diharapkan dapat memahami bagaimana sinkronisasi antar thread bekerja untuk menghindari konflik dan memastikan program berjalan secara konsisten serta terkontrol.

1.2 Tujuan Praktikum

Tujuan dari praktikum “Sistem Booking Hotel” ini adalah sebagai berikut:

1. Memahami konsep sinkronisasi antar thread pada pemrograman paralel.
2. Menerapkan penggunaan Semaphore dan Mutex dalam mengatur akses terhadap sumber daya bersama.
3. Mengidentifikasi dan mencegah terjadinya race condition pada program multithreading.
4. Menganalisis hasil eksekusi program untuk memastikan bahwa mekanisme sinkronisasi telah berjalan dengan benar.
5. Mengembangkan kemampuan mahasiswa dalam merancang, mengimplementasikan, dan menganalisis program berbasis multithreading menggunakan bahasa pemrograman C.

1.3 Rumusan Masalah

Berdasarkan latar belakang di atas, maka rumusan masalah yang diangkat dalam praktikum ini adalah sebagai berikut:

1. Bagaimana cara mengimplementasikan sistem booking hotel dengan menggunakan konsep multithreading?
2. Bagaimana cara mengatasi permasalahan race condition yang muncul saat beberapa thread mengakses sumber daya yang sama?
3. Bagaimana penggunaan semaphore dan mutex dapat membantu menjaga konsistensi data dalam sistem booking?
4. Apa yang akan terjadi jika program dijalankan tanpa mekanisme sinkronisasi?
5. Bagaimana hasil dan performa program setelah diterapkan mekanisme sinkronisasi?

2. Dasar Teori

2.1 Konsep Multithreading

Multithreading adalah suatu konsep dalam pemrograman di mana satu proses dapat menjalankan beberapa thread secara bersamaan. Thread merupakan unit terkecil dari suatu proses yang dapat dieksekusi secara independen namun tetap berbagi sumber daya dengan thread lain dalam proses yang sama.

Dengan menggunakan multithreading, program dapat melakukan lebih dari satu tugas dalam waktu yang bersamaan (concurrent execution). Misalnya, dalam simulasi sistem booking hotel, beberapa pelanggan (customer) dapat melakukan proses pemesanan kamar secara bersamaan tanpa harus menunggu pelanggan lain selesai.

Namun, karena thread-thread tersebut berbagi memori dan variabel global yang sama, dibutuhkan pengaturan yang hati-hati agar tidak terjadi konflik dalam pengaksesan data. Konflik inilah yang disebut race condition, dan untuk mencegahnya diperlukan mekanisme sinkronisasi.

2.2 Sinkronisasi dan Race Condition

Sinkronisasi adalah proses pengaturan eksekusi antar thread agar dapat berjalan secara teratur dan tidak saling mengganggu, khususnya ketika mengakses shared resource seperti variabel global, file, atau memori bersama.

Tanpa sinkronisasi, dua atau lebih thread dapat mengubah nilai variabel secara bersamaan sehingga menghasilkan hasil akhir yang tidak dapat diprediksi. Contohnya:

- Thread A membaca nilai $x = 5$, kemudian menambahnya menjadi $x = 6$.
- Pada saat yang sama, Thread B juga membaca nilai $x = 5$ dan menulis kembali menjadi $x = 10$.
Akibatnya, nilai akhir x tidak sesuai dengan urutan logika yang diharapkan.

Kondisi inilah yang disebut race condition, di mana hasil akhir program tergantung pada urutan eksekusi thread. Untuk menghindarinya, digunakan mekanisme sinkronisasi seperti mutex, semaphore, atau monitor.

2.3 Semaphore

Semaphore adalah variabel khusus yang digunakan untuk mengatur jumlah thread yang dapat mengakses sumber daya pada saat yang sama. Terdapat dua jenis semaphore utama:

1. Counting Semaphore — digunakan untuk mengatur akses ke sejumlah sumber daya terbatas (misalnya jumlah kamar hotel = 5).
2. Binary Semaphore — nilainya hanya 0 atau 1, digunakan seperti lock untuk memastikan hanya satu thread yang dapat masuk ke bagian kritis.

Operasi utama pada semaphore adalah:

- `sem_wait()` - digunakan ketika thread ingin memasuki bagian kritis (mengurangi nilai semaphore).
- `sem_post()` - digunakan ketika thread keluar dari bagian kritis (menambah nilai semaphore).

2.4 Mutex (Mutual Exclusion)

Mutex adalah singkatan dari Mutual Exclusion, yaitu mekanisme penguncian untuk memastikan hanya satu thread yang dapat menjalankan bagian tertentu dari kode pada satu waktu.

Mutex biasanya digunakan untuk melindungi bagian critical section, yaitu bagian kode yang mengakses atau memodifikasi data bersama.

Operasi dasar pada mutex:

- `pthread_mutex_lock(&lock);` - Mengunci bagian kritis, thread lain harus menunggu.
- `pthread_mutex_unlock(&lock);` - Membuka kunci agar thread lain dapat masuk.

- **2.5 Hubungan antara Semaphore dan Mutex**

- Meskipun keduanya digunakan untuk sinkronisasi, perbedaan utama antara semaphore dan mutex adalah:

Aspek	Semaphore	Mutex
Fungsi utama	Mengatur jumlah thread yang boleh mengakses resource	Mengunci akses agar hanya satu thread yang bisa masuk ke bagian kritis
Nilai awal	Bisa lebih dari satu	Hanya 0 atau 1
Kepemilikan	Tidak memiliki konsep kepemilikan	Hanya thread yang mengunci yang boleh membuka kembali
Contoh kasus	Jumlah kamar hotel, jumlah koneksi	Pengaturan print output, update variabel bersama

- Dengan menggabungkan keduanya, sistem dapat mengatur kuantitas akses (semaphore) dan ketertiban akses (mutex) secara bersamaan, sehingga program multithreading dapat berjalan aman dan konsisten.

3. Soal Pemrograman: Sistem Booking Hotel

3.1. Analisis Masalah

Pemahaman tentang Soal

Program ini bertujuan untuk membuat simulasi sistem booking hotel dengan kondisi:

- Tersedia 5 kamar hotel yang dapat di-booking.
- Terdapat 10 customer yang mencoba melakukan booking secara bersamaan (multithreading).
- Setiap customer akan mencoba melakukan booking kamar, menunggu beberapa waktu, lalu melakukan check-out sehingga kamar bisa digunakan kembali.

Tujuan utama dari simulasi ini adalah untuk memahami bagaimana sinkronisasi antar thread dapat mencegah terjadinya konflik (race condition) ketika beberapa thread mengakses sumber daya yang sama (dalam hal ini jumlah kamar).

Identifikasi Masalah Sinkronisasi

Masalah utama yang mungkin muncul adalah:

- Beberapa thread (customer) mencoba memesan kamar pada saat yang bersamaan.
- Jika tidak ada pengaturan sinkronisasi, bisa terjadi kondisi race condition, di mana jumlah kamar yang tersedia bisa menjadi tidak akurat (misalnya terhitung negatif atau lebih dari kapasitas sebenarnya).
- Selain itu, hasil output bisa saling tumpang tindih di terminal.

Desain Solusi

Untuk mengatasi hal tersebut, digunakan dua mekanisme sinkronisasi utama:

1. Semaphore

Digunakan untuk mengatur jumlah kamar hotel yang tersedia. Setiap kali customer berhasil booking, semaphore akan berkurang. Saat check-out, semaphore bertambah lagi.

2. Mutex (Mutual Exclusion)

Digunakan untuk melindungi bagian critical section yang berhubungan dengan output printing agar tampilan hasil tidak saling bertumpuk.

Selain itu, setiap customer akan mencoba booking hingga maksimal **3 kali**. Jika kamar penuh pada percobaan ke-3, customer dianggap menyerah.

3.2. Implementasi

Penjelasan Algoritma

1. Inisialisasi semaphore dengan nilai 5 (menandakan jumlah kamar tersedia).
2. Membuat 10 thread customer.
3. Setiap customer akan:
 - Mencoba booking kamar (dengan waktu tunggu acak 1–3 detik).
 - Jika berhasil, tidur selama 5 detik (simulasi menginap).
 - Setelah itu, melakukan check-out dan mengembalikan kamar ke sistem.
 - Jika gagal booking (karena kamar penuh), menunggu 2 detik lalu mencoba lagi.
 - Jika sudah mencoba 3 kali dan tetap gagal, maka customer menyerah.
4. Semua thread dijalankan secara bersamaan dan saling bersinkronisasi menggunakan semaphore dan mutex.

3.3. Testing dan Output

Test Case yang Diuji

- 10 customer mencoba booking secara bersamaan.
- Ada customer yang gagal booking karena kamar penuh.
- Ada customer yang berhasil setelah mencoba beberapa kali.
- Semua kamar kembali tersedia setelah seluruh customer check-out.

Hasil yang Didapat

- Output berjalan sesuai ekspektasi.
- Tidak ada data kamar yang terhitung negatif atau lebih dari kapasitas.
- Urutan output tetap rapi dan mudah dibaca.

3.4. Analisis Hasil

Apakah program berjalan sesuai ekspektasi?

Ya. Program berhasil menjaga agar jumlah kamar tidak pernah melebihi batas 5, serta output yang ditampilkan tetap teratur walaupun banyak thread berjalan bersamaan.

Masalah yang Ditemui dan Solusinya

- **Masalah:** Output kadang saling menimpa jika tidak menggunakan mutex.
Solusi: Ditambahkan `pthread_mutex_lock()` dan `pthread_mutex_unlock()` di bagian `printf()`.
- **Masalah:** Beberapa thread gagal booking karena kamar penuh.
Solusi: Diberikan sistem retry maksimal 3 kali agar simulasi lebih realistis.

Performa Program

Program berjalan dengan baik pada sistem multithreading. Waktu eksekusi bergantung pada jumlah thread dan delay yang diberikan (sleep time).

Pelajaran yang Didapat

Dari percobaan ini, dapat dipahami bahwa:

- Sinkronisasi sangat penting dalam pemrograman paralel/multithreading.
- Tanpa sinkronisasi, hasil program bisa salah dan tidak konsisten.
- Pemakaian mutex dan semaphore secara tepat dapat menghindari race condition serta membuat sistem lebih stabil dan akurat.

1. Kode Source sistem booking hotel

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>
#include <unistd.h>
#include <stdarg.h>

#define JUMLAH_KAMAR 5
#define JUMLAH_CUSTOMER 10
#define MAKS_COBA 3

sem_t semaphore_kamar;
pthread_mutex_t mutex_print;
int kamar_terpakai = 0;

void now_str(char *buf, size_t n) {
    time_t t = time(NULL);
    struct tm tm = *localtime(&t);
    snprintf(buf, n, "%02d:%02d:%02d", tm.tm_hour, tm.tm_min, tm.tm_sec);
}

void log_pesan(const char *fmt, ...) {
    va_list ap;
    char waktu[16];
    now_str(waktu, sizeof(waktu));
    pthread_mutex_lock(&mutex_print);
    printf("[%s] ", waktu);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    printf("\n");
    fflush(stdout);
    pthread_mutex_unlock(&mutex_print);
}
```

```

void *customer_thread(void *arg) {
    int id = *(int *)arg;
    free(arg);
    unsigned int seed = (unsigned int)time(NULL) ^ (unsigned int)id;

    for (int percobaan = 1; percobaan <= MAKS_COBA; percobaan++) {
        log_pesan("  Customer-%d mencoba booking (percobaan ke-%d)...", id, percobaan);
        int delay = (rand_r(&seed) % 3) + 1;
        sleep(delay);

        if (sem_trywait(&semaphore_kamar) == 0) {
            pthread_mutex_lock(&mutex_print);
            kamar_terpakai++;
            int sisa = JUMLAH_KAMAR - kamar_terpakai;
            pthread_mutex_unlock(&mutex_print);

            log_pesan("✅ Customer-%d berhasil booking! (%d kamar tersisa)", id, sisa);
            sleep(5);

            pthread_mutex_lock(&mutex_print);
            kamar_terpakai--;
            int sisa_setelah = JUMLAH_KAMAR - kamar_terpakai;
            pthread_mutex_unlock(&mutex_print);

            sem_post(&semaphore_kamar);
            log_pesan("👤 Customer-%d check-out (%d kamar tersisa)", id, sisa_setelah);
            return NULL;
        } else {
            log_pesan("❌ Customer-%d gagal booking (penuh), mencoba lagi setelah 2 detik...", id);
            sleep(2);
        }
    }

    log_pesan("😓 Customer-%d menyerah setelah %d kali mencoba.", id, MAKS_COBA);
    return NULL;
}

```

```

int main(void) {
    pthread_t threads[JUMLAH_CUSTOMER];
    sem_init(&semaphore_kamar, 0, JUMLAH_KAMAR);
    pthread_mutex_init(&mutex_print, NULL);

    log_pesan("=== SISTEM BOOKING HOTEL ===");
    log_pesan("Total Kamar: %d\n", JUMLAH_KAMAR);

    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        int *id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&threads[i], NULL, customer_thread, id);
        sleep(1);
    }

    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        pthread_join(threads[i], NULL);
    }

    log_pesan("\n=== Semua customer selesai ===");
    sem_destroy(&semaphore_kamar);
    pthread_mutex_destroy(&mutex_print);
    return 0;
}

```

2. Output

```
root@RIDHO:~# nano booking_hotel.c
root@RIDHO:~# gcc -std=c11 -pthread -o booking_hotel booking_hotel.c
root@RIDHO:~# ./booking_hotel
[02:12:20] === SISTEM BOOKING HOTEL ===
[02:12:20] Total Kamar: 5

[02:12:20] 🧑 Customer-1 mencoba booking (percobaan ke-1)...
[02:12:21] 🧑 Customer-2 mencoba booking (percobaan ke-1)...
[02:12:22] ✅ Customer-1 berhasil booking! (4 kamar tersisa)
[02:12:22] 🧑 Customer-3 mencoba booking (percobaan ke-1)...
[02:12:23] 🧑 Customer-4 mencoba booking (percobaan ke-1)...
[02:12:24] ✅ Customer-3 berhasil booking! (3 kamar tersisa)
[02:12:24] ✅ Customer-2 berhasil booking! (2 kamar tersisa)
[02:12:24] 🧑 Customer-5 mencoba booking (percobaan ke-1)...
[02:12:25] ✅ Customer-4 berhasil booking! (1 kamar tersisa)
[02:12:25] 🧑 Customer-6 mencoba booking (percobaan ke-1)...
[02:12:26] 🧑 Customer-7 mencoba booking (percobaan ke-1)...
[02:12:27] 🧑 Customer-1 check-out (2 kamar tersisa)
[02:12:27] ✅ Customer-5 berhasil booking! (1 kamar tersisa)
[02:12:27] 🧑 Customer-8 mencoba booking (percobaan ke-1)...
[02:12:28] ✅ Customer-6 berhasil booking! (0 kamar tersisa)
[02:12:28] 🧑 Customer-9 mencoba booking (percobaan ke-1)...
[02:12:29] 🧑 Customer-3 check-out (1 kamar tersisa)
[02:12:29] 🧑 Customer-2 check-out (2 kamar tersisa)
[02:12:29] ✅ Customer-7 berhasil booking! (1 kamar tersisa)
[02:12:29] ✅ Customer-8 berhasil booking! (0 kamar tersisa)
[02:12:29] 🧑 Customer-10 mencoba booking (percobaan ke-1)...
[02:12:30] 🧑 Customer-4 check-out (1 kamar tersisa)
[02:12:30] ✅ Customer-9 berhasil booking! (0 kamar tersisa)
[02:12:32] 🧑 Customer-5 check-out (1 kamar tersisa)
[02:12:32] ✅ Customer-10 berhasil booking! (0 kamar tersisa)
[02:12:33] 🧑 Customer-6 check-out (1 kamar tersisa)
[02:12:34] 🧑 Customer-7 check-out (2 kamar tersisa)
[02:12:34] 🧑 Customer-8 check-out (3 kamar tersisa)
[02:12:35] 🧑 Customer-9 check-out (4 kamar tersisa)
[02:12:37] 🧑 Customer-10 check-out (5 kamar tersisa)
[02:12:37]
=== Semua customer selesai ===
```

3. Penjelasan bagaimana Anda mengatasi race condition

Race condition terjadi ketika dua atau lebih thread atau process mengakses dan memodifikasi data yang sama secara bersamaan, sehingga hasil akhirnya menjadi tidak terduga.

Untuk mengatasinya, digunakan mekanisme sinkronisasi agar hanya satu thread yang dapat mengakses data kritis pada satu waktu. Beberapa cara yang umum digunakan:

1. Menggunakan Mutex (Mutual Exclusion)

- Mutex berfungsi sebagai “kunci”.
- Saat satu thread sedang mengakses variabel bersama (shared resource), thread lain harus menunggu hingga kunci dilepas.
- Contoh:
 - `pthread_mutex_t lock;`
 - `pthread_mutex_init(&lock, NULL);`
 - `pthread_mutex_lock(&lock);`
 - `// Bagian kode kritis (critical section)`
 - `pthread_mutex_unlock(&lock);`

2. Menggunakan Semaphore

- Semaphore bekerja seperti sinyal untuk mengatur berapa banyak thread yang boleh mengakses resource pada saat bersamaan.
- Biasanya digunakan jika ada lebih dari satu sumber daya yang bisa diakses.

3. Menggunakan Monitor atau Condition Variable

- Dipakai untuk mengatur urutan eksekusi antar thread (misalnya menunggu suatu kondisi terpenuhi dulu).

Jadi, inti solusi dari race condition adalah menggunakan mekanisme sinkronisasi (mutex, semaphore, dsb) agar thread tidak saling berebut resource yang sama.

4. Analisis: Apa yang terjadi jika tidak menggunakan sinkronisasi

Jika tidak ada sinkronisasi, maka:

1. Race condition pasti terjadi.
 - Dua atau lebih thread bisa mengubah variabel global pada saat bersamaan.
 - Akibatnya hasil perhitungan bisa berbeda-beda setiap kali program dijalankan.
2. Data menjadi tidak konsisten (data corruption).
 - Misalnya ada variabel saldo = 1000; Dua thread ingin menambah 100 dan 200 secara bersamaan. Tanpa sinkronisasi, hasil akhirnya bisa tetap 1000, 1100, atau 1200 secara acak — padahal seharusnya 1300.
3. Program menjadi tidak deterministik.
 - Sulit dideteksi bug-nya karena tidak selalu muncul.
 - Hasil output tergantung pada kecepatan eksekusi CPU atau urutan thread berjalan.
4. Potensi crash atau deadlock tak terkontrol.
 - Jika dua thread menulis pada memori yang sama tanpa aturan, bisa menyebabkan crash atau error memori.

4. Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan hasil praktikum dan analisis yang telah dilakukan pada tugas Sistem Booking Hotel, dapat disimpulkan bahwa:

1. Mekanisme sinkronisasi sangat penting dalam pemrograman multithreading untuk mencegah terjadinya race condition dan menjaga konsistensi data ketika beberapa thread mengakses sumber daya yang sama secara bersamaan.
2. Semaphore berfungsi efektif untuk membatasi jumlah thread yang dapat mengakses sumber daya tertentu, seperti jumlah kamar hotel yang tersedia dalam simulasi ini.
3. Mutex digunakan untuk memastikan hanya satu thread yang dapat mengakses bagian kritis program pada satu waktu, misalnya saat proses pencetakan output, sehingga hasil tidak tumpang tindih.
4. Dengan penerapan kombinasi semaphore dan mutex, sistem booking hotel dapat berjalan secara teratur, adil, dan bebas dari kesalahan perhitungan jumlah kamar.
5. Program yang dijalankan menunjukkan hasil yang deterministik dan stabil, di mana jumlah kamar tidak pernah melebihi batas kapasitas dan urutan output tetap konsisten meskipun proses dijalankan secara paralel.
6. Praktikum ini memberikan pemahaman mendalam mengenai bagaimana sinkronisasi antar thread dapat diterapkan dalam kasus nyata untuk mengontrol perilaku proses yang berjalan bersamaan.

4.2 Saran

Berdasarkan pelaksanaan praktikum dan hasil yang diperoleh, beberapa saran yang dapat diberikan adalah sebagai berikut:

1. Optimasi penggunaan thread dapat dilakukan dengan menyesuaikan jumlah thread terhadap kapasitas sumber daya agar sistem tidak mengalami beban berlebih.
2. Untuk pengembangan lebih lanjut, program dapat dikembangkan dengan fitur tambahan, seperti sistem antrian pelanggan atau prioritas booking, guna meniru kondisi sistem nyata.
3. Disarankan untuk menggunakan struktur data yang lebih kompleks, seperti queue atau linked list, untuk mengatur status kamar dan pelanggan agar simulasi lebih realistis.
4. Pengujian dapat diperluas menggunakan jumlah thread dan resource yang lebih besar untuk mengamati performa dan skalabilitas sistem sinkronisasi.
5. Dalam praktik di dunia nyata, pemahaman mengenai sinkronisasi harus terus dikembangkan karena konsep ini juga digunakan dalam berbagai bidang seperti sistem operasi, jaringan, dan pemrograman paralel.

5. Daftar Pustaka

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th Edition). Wiley.
 - Digunakan sebagai dasar teori mengenai konsep sinkronisasi, semaphore, mutex, dan race condition pada sistem operasi multithreading.
2. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th Edition). Pearson Education.
 - Menjelaskan prinsip dasar proses dan thread, serta metode sinkronisasi antar proses pada sistem multiprosesor.
3. Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th Edition). Pearson.
 - Menjadi referensi tambahan dalam memahami mekanisme penguncian (locking) dan pengaturan concurrency.
4. GNU Project. (2024). POSIX Threads Programming (pthreads) — GNU C Library Documentation.
Diakses dari:
https://www.gnu.org/software/libc/manual/html_node/Pthreads.html
 - Digunakan sebagai referensi utama dalam implementasi fungsi-fungsi thread dan mutex seperti `pthread_create`, `pthread_join`, `pthread_mutex_lock`, dan `pthread_mutex_unlock`.
5. Open Group. (2024). POSIX Semaphores Overview.
Diakses dari:
https://pubs.opengroup.org/onlinepubs/9699919799/functions/sem_init.html
 - Referensi resmi mengenai penggunaan semaphore pada bahasa C, termasuk fungsi `sem_init`, `sem_wait`, dan `sem_post`.
6. GeeksforGeeks. (2023). Semaphore vs Mutex in Operating System.
Diakses dari: <https://www.geeksforgeeks.org/semaphore-vs-mutex/>
 - Menjadi referensi tambahan untuk penjelasan perbandingan antara semaphore dan mutex pada bagian dasar teori laporan.
7. Tutorialspoint. (2023). C - Multithreading.
Diakses dari:
https://www.tutorialspoint.com/cprogramming/c_multithreading.htm
 - Digunakan sebagai referensi dalam penulisan kode program C untuk implementasi thread menggunakan pustaka pthread.

